

OrbitGuard

Implementation Plan

Free, explainable, validated conjunction screening for small operators

Empty repo → polished, TraCSS-validated, demoable product

WHAT THIS PLAN COMMITS TO

- Screening sieve with a provable no-miss guarantee (padded coarse grid + KD-tree + Brent TCA)
 - Real 2D-Pc (Foster + Chan), unit-tested against NASA CARA's published test cases
 - PcMax worst-case bound when covariance is absent — TLE data proves safety, never danger
 - Validation scorecard vs the official U.S. TraCSS answer key (recall / precision, published)
 - Groq-narrated explanations under a strict narrate-only contract (\$0 LLM budget)
 - Live CesiumJS 3D globe + Encounter Inspector with the covariance-ellipse money shot
- Sections: architecture · data · algorithms (math) · validation · phases · team · demo · risks · references

\$0 budget

Software-only

Team of 4-5

Resume-first

OrbitGuard — Implementation Plan

****Free, explainable, *validated* satellite conjunction screening for small operators****

FAR AWAY 2026 · Space & Aerospace · Team of 4–5 · \$0 budget · Prepared by Fable 5 (lead architect) · 12 June 2026

1. Executive summary

OrbitGuard is a free, browser-based conjunction-assessment dashboard. It ingests public orbital data, screens a small operator's satellites against the full public catalog for close approaches, computes a collision-risk figure with the **actual operational math** (2D probability of collision in the encounter plane), renders everything on a live 3D globe, and issues a clear, *explained* **DODGE / WAIT / WATCH** triage call. Its accuracy is not asserted — it is **measured against the official U.S. TraCSS "Dataset for Conjunction Assessment Verification" answer key** and published as a scorecard inside the app.

The deep research changed five things from our draft, all for the better:

#	Decision	Our draft said	This plan says	Why
1	Validation input	"validate vs TraCSS key" (vaguely)	TraCSS provides ephemerides (20.7 GB), not TLEs — we run our screening engine directly on them	Decouples screening-algorithm validation from TLE error; far stronger claim
2	Collision probability	"simplified estimate"	Real 2D-Pc (Foster 1992) + Chan (1997) analytic cross-check, unit-tested against NASA CARA's open-source test cases ; PcMax bound when covariance is absent	"We match NASA's reference implementation" is an interview-winning sentence
3	Risk policy	generic DODGE/WAIT/WATCH	Asymmetric, operationally honest policy: TLE-only data can prove safety, never danger (PcMax small → safe; PcMax large → escalate to CDM)	Deep, defensible, memorable
4	Data format	TLEs, "use OMM for new objects"	OMM-first (JSON/CSV) everywhere; 5-digit catalog numbers exhaust at 69999 ~12 July 2026 — possibly mid-event	Verified on CelesTrak; legacy-TLE-only pipelines break this summer
5	LLM explainer	claude-opus-4-8	Groq free tier, llama-3.3-70b-versatile (30 req/min, 1K req/day, 100K tok/day — verified), strict narrate-only contract + template fallback	\$0 constraint; LLM is a thin layer here by design

Product scoping insight: a small operator does not need all-vs-all screening of 30,000 objects. They need **their k satellites vs everything** — an $O(k \cdot N)$ problem that runs in seconds on a laptop. That is the product. Catalog-wide all-vs-all (SOCRATES-style) becomes an overnight batch stretch goal, not a blocking dependency.

Headline we will be able to defend: *"OrbitGuard reproduces the official TraCSS answer-key conjunctions at X% recall / Y% precision, computes collision probability that matches NASA CARA's reference implementation on its published test cases, and explains every triage call in plain language — for free, in a browser."*

2. Prior art — what exists, and our wedge

Tool	What it does	What it lacks (our wedge)
SOCRATES Plus (CelesTrak)	Batch conjunction reports 3×/day, all active payloads vs full catalog, 7-day lookahead, min-range + max-Pc tables	Table-only UI, no 3D, no explanation, no triage guidance, no published validation scorecard
Privateer Wayfinder	Beautiful near-real-time CesiumJS visualization of tracked objects	Visualization-first; no open, validated screening pipeline; no explainable triage
KeepTrack.space	Open-source 3D SSA toolkit, 30k+ objects in browser, breakup/launch sims	Analyst toolkit, not a validated CA pipeline; no Pc math validated against a reference
LeoLabs, SpaceX Starling/ops, COMSPOC	Commercial-grade SSA, proprietary catalogs and screening	Costs lakhs/year; closed; inaccessible to university/small/Indian operators

Tool	What it does	What it lacks (our wedge)
NASA CARA Analysis Tools (GitHub)	Open MATLAB SDKs for Pc, max-Pc, covariance tools, with test cases and expected results	Algorithms only — no product, no pipeline, no UI. We port, cite, and test against them
TraCSS (Office of Space Commerce)	The forthcoming U.S. civil space-traffic system; published the verification dataset	TraCSS serves U.S.-coordinated operators; our niche is the long tail, and we <i>use</i> their dataset as our exam

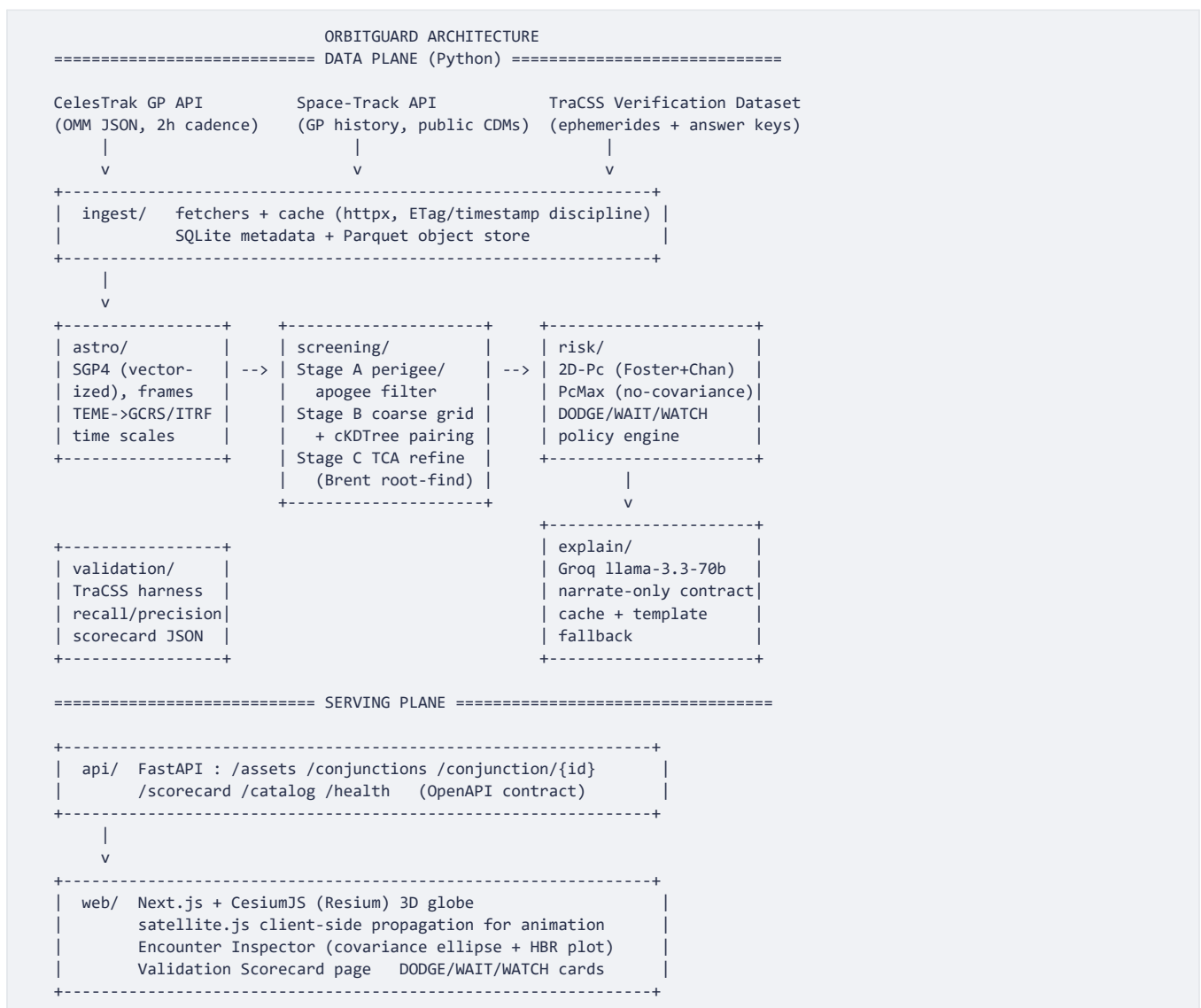
Positioning sentence: Wayfinder's looks + SOCRATES' screening + NASA CARA's math, with an explanation layer none of them have — validated in the open, free for the operators that commercial SSA prices out (university cubesats, Indian smallsat startups like Pixxel/GalaxEye).

Two findings from the literature that shape the architecture:

TLEs must be propagated with SGP4 and nothing else. TLEs are *mean* elements fitted to the SGP4 theory; feeding them to a numerical propagator is a category error. Accuracy is ~1 km at epoch, degrading 1–3 km/day (in-track dominant, up to ~25 km after a few days). This bounds what TLE-only screening can honestly claim — and drives decision #3 above.

poliastro is archived (Oct 2023). Our draft's implied alternatives are stale. The maintained Python stack is python-sgp4 (vectorized C++ core) + skyfield (time scales, frames). Orekit remains the heavyweight option only if we later ingest operator ephemerides (stretch).

3. System architecture



Data-flow rule that keeps us honest: **every number shown in the UI is computed in the data plane; the LLM only converts an already-final JSON risk record into prose.** The LLM cannot change a recommendation, a probability, or a distance.

4. Data acquisition plan

4.1 Sources (all free, all verified live in June 2026)

Source	What we take	Endpoint / access	Discipline
CelesTrak GP	Full catalog + groups as OMM JSON/CSV	<code>celestrak.org/NORAD/elements/gp.php?GROUP=active&FORMAT=json</code>	Data refreshes every 2 h — never poll faster; ~100 MB/day bandwidth cap and one-download-per-update enforced; violators get IP-blocked. We cache every snapshot
Space-Track	GP/GP_History (OMM), public CDMs with covariance	REST API after free account login	Throttle per their docs; nightly batch only; cache everything
TraCSS dataset	AerospaceIIVDataset_20251009a.tar.gz (20.73 GB ephemerides), spherical answer key (..._Spherical_DefaultHBR.csv.gz), SFSH rectangular key, volume mappings, User's Guide PDF	Google account; Google Form gives immediate access (Drive links need manual approval); CC0 license	Download once to one machine; convert to Parquet; share via external drive/LAN, not re-downloads
SOCRATES Plus	Independent conjunction lists	<code>celestrak.org/SOCRATES</code>	Used as a sanity cross-check oracle for live-mode screening
Groq API	LLM explanations	OpenAI-compatible REST, llama-3.3-70b-versatile	30 RPM / 1K RPD / 12K TPM / 100K TPD (verified); caching + fallback below
Cesium ion	Globe imagery/terrain token	Free community tier	One token, restricted to our domains

4.2 Format policy — OMM-first (hard requirement)

The 5-digit NORAD catalog space effectively exhausts at **69999**, **estimated ~12 July 2026** (70000+ is reserved for analyst objects); new objects then receive 6-digit numbers that the legacy TLE format **cannot represent**. CelesTrak already defaults to OMM-based CSV (May 2026). Therefore:

Internal canonical record = CCSDS OMM fields (JSON), catalog ID stored as integer, never as a 5-char string.

Legacy TLE accepted at the edge for user-pasted input only, converted immediately.

satellite.js on the client is fed OMM-derived parameters, not raw TLE lines, wherever possible.

This is a judge-visible differentiator: most hobby tools break this summer; ours won't, by design.

4.3 Day-zero actions (literally before any code)

1. Submit the **TraCSS Google Form** (immediate-access path) with a team Google account; also request the Drive links as backup. Contact if stuck: Anna.Parikka@noaa.gov.
2. Create **Space-Track.org** account; read API throttle rules; store creds in `.env`.
3. Create **Groq** API key; confirm llama-3.3-70b-versatile quota on the console.
4. Create **Cesium ion** free token.
5. Pull and cache one full **CelesTrak** active **group snapshot** (OMM JSON) → commit a frozen copy under `data/fixtures/` (small subset) for deterministic tests and the offline demo mode.
6. Start the 20.73 GB TraCSS download on the best connection available; verify checksums; begin tar→Parquet conversion overnight.

7. Pick the **demo asset set**: 3–5 satellites with an India story (e.g., a Pixxel Firefly, a GalaxEye demo sat if cataloged, an ISRO payload, plus ISS for audience recognition).

5. Algorithm design

This section is the technical heart. Each subsection states the chosen method, the actual math, and why it beats the alternatives we researched.

5.1 Propagation, time, and frames

Choice: `python-sgp4` (**vectorized** `SatrecArray`) for all GP propagation; `Skyfield` for time scales and frame rotations. Numerical propagators rejected for GP data.

TLE/OMM GP elements are *mean elements fitted to SGP4*; propagating them with anything else (Orekit numerical, hapsira/poliastro Cowell) silently de-calibrates them. This is a correctness point many "globe demos" get wrong.

`python-sgp4` exposes a C++-accelerated `SatrecArray`: one call propagates the whole catalog over a time grid into numpy arrays — the enabler for our vectorized screening (Section 5.2).

SGP4 outputs are in **TEME** (True Equator, Mean Equinox). Mixing TEME with J2000/GCRF without rotation injects hundreds of metres to kilometres of error. All frame conversions go through one audited module: TEME → GCRS (for inter-satellite geometry near an epoch, TEME-vs-TEME differences cancel for *relative* positions of co-epoch states, but we convert once, centrally, anyway) and GCRS → ITRF (for the globe). Time handling: UTC in/out, TT/UT1 internally via `Skyfield`'s timescale; leap seconds come from `Skyfield`'s bundled data.

Error model (used by the risk layer): GP accuracy ≈ 1 km at epoch, growing 1–3 km/day, in-track dominant. We encode a conservative diagonal "TLE-grade pseudo-covariance" only for *labeling* purposes (data-quality badge), never for a claimed rigorous Pc.

Golden tests: propagate fixture satellites and assert against independently generated `Skyfield` positions and published SGP4 verification vectors (Vallado's test cases ship with the `sgp4` package) to catch frame/time regressions in CI.

5.2 Conjunction screening — the three-stage sieve

Problem: k assets vs $N \approx 30,000$ catalog objects over a 7-day window, without missing any approach closer than the screening distance D (we adopt $D = 10$ km LEO default, configurable; TraCSS runs use the volumes the answer key prescribes).

Stage A — apogee/perigee gate (Hoots Filter I). Discard any pair whose altitude bands can't intersect within pad:

```
reject if max(q_i, q_j) - min(Q_i, Q_j) > D_pad
where q = perigee radius, Q = apogee radius, D_pad = D + drift margin (default 25 km)
```

$O(N)$ per asset, eliminates the majority of the catalog for typical LEO assets (e.g., a 550-km asset never meets a GEO bird). The drift margin absorbs perigee/apogee evolution over the window (drag, J2 on eccentricity) — classic Hoots Filter I with a safety pad, per the AGI filters paper.

Stage B — coarse time grid + spatial pairing (the "smart sieve" guarantee). For survivors, propagate everything (vectorized) on a coarse grid with step Δt and flag any pair whose separation at any grid time is below a **padded radius**:

```
R_pair(Δt) = D + v_rel,max * Δt / 2      (+ small acceleration term, <1 km, absorbed in pad)
LEO worst case: v_rel,max ≈ 2 * v_circ ≈ 15.6 km/s
```

Why the pad is a guarantee, not a heuristic: if a conjunction with miss distance $< D$ occurs at time t , *the nearest grid point is within $\Delta t/2$ of t* , and the pair can have separated by at most $v_{rel,max} \cdot \Delta t/2$ — so its grid-time separation is $< D + v_{rel,max} \cdot \Delta t/2 = R_{pair}$. No true conjunction can escape the coarse net. This is exactly the sieve logic of Alarcón-Rodríguez et al. (2002), which ESA built CRASS on; we implement it with modern vectorized tooling:

Default $\Delta t = 30$ s $\rightarrow R_{pair} \approx 244$ km.

Per grid step, build a `scipy.spatial.cKDTree` over catalog positions and range-query each asset (k assets $\rightarrow k$ queries/step): $O(N \log N)$ per step instead of $O(N^2)$.

Memory discipline: time-chunked propagation (e.g., 2-hour blocks), float32 position buffers. Asset-vs-catalog over 7 days at 30 s = 20,160 steps; with ~3–6k Stage-A survivors this runs in tens of seconds on a laptop — measured and reported in the funnel panel.

Hits open a $\pm\Delta t$ window around each flagged grid time and pass to Stage C.

Why this beats the classical alternative: Hoots Filters II (orbit-path) and III (time) assume near-Keplerian geometry and need careful handling for perturbed/decaying orbits and edge cases (near-circular co-planar pairs). The padded coarse-grid sieve is brute-force-with-a-proof: simpler to verify, trivially parallel, and its cost is already acceptable at our $k \cdot N$ scale. We cite both and *explain this trade* in the README — judges who know the field will recognize the literacy.

Stage C — TCA refinement. Within each flagged window, define

$$f(t) = |\Delta r(t)|^2 \quad g(t) = f'(t) = 2 \Delta r(t) \cdot \Delta v(t)$$

Sample g at fine step (1 s), bracket sign changes ($- \rightarrow +$), then Brent root-finding on $g(t)$ to machine-precision TCA; evaluate miss distance $d = |\Delta r(\text{TCA})|$ and relative velocity v_{rel} at TCA. Deduplicate events closer than 60 s. Numerical cost is trivial (dozens of evaluations per event). Targets: TCA error < 10 ms numerical (physics error is TLE-dominated and separately disclosed); miss-distance numerical error < 1 m.

The funnel itself is a demo asset: the UI shows 30,000 objects $\rightarrow$ Stage A: 4,213 $\rightarrow$ Stage B: 57 windows $\rightarrow$ Stage C: 9 conjunctions $\rightarrow$ top risk: 1, with timings. That one strip communicates more engineering than any paragraph.

5.3 Collision probability — the 2D-Pc encounter-plane method

Choice: Foster (1992) 2D numerical integration as the reference implementation (NASA CARA's operational method), Chan (1997) analytic series as the fast cross-check, both unit-tested against NASA CARA's published test cases. PcMax bound when covariance is unavailable.

Setup (short-encounter assumptions). At TCA, relative velocity is high (LEO conjunctions are typically km/s); the encounter is effectively instantaneous, relative motion is linear through the encounter, and position errors are Gaussian with negligible velocity-error contribution. Then the 3D problem collapses to 2D in the **encounter plane** (B-plane) perpendicular to v_{rel} :

1. Combined covariance $C = C_1 + C_2$ (3×3 , both rotated to a common frame at TCA).
2. Build encounter-plane basis $\{e_1, e_2\}$ perpendicular to v_{rel} ; project: $C_p = E^T C E$ (2×2), $m = E^T \Delta r$ (the projected miss vector).
3. Both objects are conservatively merged into one **hard-body radius** R (sum of bounding-sphere radii; TraCSS keys prescribe HBR per object class — "DefaultHBR" / "DiscreteHBR").

The probability is a 2D Gaussian integrated over the hard-body disk:

$$P_c = (1 / (2\pi \sqrt{\det C_p})) \iint_{\{|x| \leq R\}} \exp(-(x - m)^T C_p^{-1} (x - m) / 2) dA$$

Foster method: evaluate the integral numerically in polar coordinates; the radial inner integral reduces to error-function terms, leaving a 1D angular quadrature — robust over the full operational range ($P_c 10^{-1}$ to 10^{-7} , where all mainstream methods agree). This is what NASA CARA runs operationally; we port `Pc2D_Foster` from the open `nasa/CARA_Analysis_Tools` SDK (MATLAB $\rightarrow$ Python) **and port its bundled test cases with expected results into our pytest suite**. Passing those tests is our correctness certificate — and a one-line resume claim.

Chan method (cross-check + speed): Chan converts the integral to a Rician series via an equivalent-area circle. With principal-axis standard deviations σ_x, σ_y and miss components (m_x, m_y) :

$$\begin{aligned} u &= R^2 / (\sigma_x \sigma_y) && \text{(object-size term)} \\ v &= m_x^2 / \sigma_x^2 + m_y^2 / \sigma_y^2 && \text{(Mahalanobis miss term)} \\ P_c &\approx e^{-v/2} \sum_{m=0..M} (v^m / (2^m m!)) [1 - e^{-u/2} \sum_{k=0..m} (u^k / (2^k k!))] \end{aligned}$$

Three terms suffice in the operational regime (per the published method comparisons; Chan is orders of magnitude faster than Foster, Foster is the accuracy reference). Every UI P_c displays the Foster value; CI asserts $|Foster - Chan|$ within tolerance on the test corpus — disagreement flags a geometry bug before it ships.

Where the covariance comes from:

CDM-grade: Space-Track public CDMs carry both objects' covariance (CCSDS CDM standard; TraCSS CDM spec v2.1). When a CDM exists for our pair, we compute rigorous P_c and badge the result CDM-GRADE.

TLE-grade (no covariance): we do **not** invent a covariance. We compute the **maximum possible P_c** over all covariances consistent with the miss geometry. For the isotropic case the bound has a clean closed form — with $R \ll d$ and σ the unknown isotropic deviation:

$$P_c(\sigma) \approx (R^2 / 2\sigma^2) \cdot \exp(-d^2 / 2\sigma^2) \quad \text{maximize over } \sigma: d/d\sigma P_c = 0 \Rightarrow \sigma^2 = d^2/2$$

$$\Rightarrow P_{cMax} = R^2 / (e \cdot d^2)$$

The general anisotropic maximization (covariance scaled and oriented worst-case, miss vector on the major axis) follows Alfano (2014)/Frisbee (2015) and NASA CARA's "single covariance max P_c " SDK, which we port for the rigorous version. Result badge: TLE-GRADE (worst-case bound).

The asymmetry that defines our risk policy: P_{cMax} is an upper bound, so it can **clear** an event ($P_{cMax} < \text{threshold} \rightarrow$ safe under *any* consistent covariance) but can never **convict** one. TLE-only data can prove safety, never danger. This single sentence is the most defensible piece of design in the product, and it is exactly how a real operator should treat public-catalog warnings.

Known limits (disclosed in-app, scoped out): very low relative-velocity encounters (GEO station-keeping neighbors, formation flying) violate the short-encounter assumptions and need 3D/ nonlinear P_c (Coppola); we detect $v_{rel} < 100$ m/s and label such events "2D- P_c not applicable" rather than printing a wrong number. Honesty beats coverage.

5.4 Risk policy — DODGE / WAIT / WATCH

Thresholds anchor to the NASA CA Handbook practice ($P_c \geq 10^{-4}$ triggers maneuver consideration; mitigation aims ~1.5 orders of magnitude lower):

Data grade	Condition	Verdict	Rationale shown to user
CDM-grade	$P_c \geq 10^{-4}$	DODGE	Above NASA maneuver-consideration threshold
CDM-grade	$10^{-6} \leq P_c < 10^{-4}$	WATCH	Within monitoring band; re-screen on every data update
CDM-grade	$P_c < 10^{-6}$	WAIT	Below concern band
TLE-grade	$P_{cMax} < 10^{-6}$	WAIT	Safe under <i>any</i> consistent covariance (provable)
TLE-grade	$P_{cMax} \geq 10^{-4}$ and $d < \text{escalation bound}$	WATCH + ESCALATE	TLE data cannot justify a maneuver; instruction: request CDM / owner ephemeris
TLE-grade	otherwise	WATCH	Inconclusive; monitor next GP update
any	$v_{rel} < 100$ m/s	WATCH + flag	2D- P_c invalid; needs high-fidelity analysis

Every verdict carries its full evidence record: TCA (UTC), miss distance with radial/in-track/ cross-track decomposition, v_{rel} , P_c or P_{cMax} with method name, data grade, screening volume, and data ages of both objects. That record — not free text — is what the LLM receives.

5.5 Explainability layer (Groq)

Contract: the LLM is a *narrator*. Input = the final JSON evidence record (numbers already decided). Output = 4–6 sentences: what happens, when, how close, why the verdict follows from the thresholds, what to do next. The system prompt forbids introducing numbers not present in the input; a post-validator rejects any output whose digits don't appear in the evidence record (simple regex digest match) and falls back to templates.

Wiring & budget (verified limits: 30 RPM / 1K RPD / 12K TPM / 100K TPD):

Model llama-3.3-70b-versatile via the OpenAI-compatible endpoint; static system prompt (cached by Groq, not counted toward token limits per their docs).

~700 input + ~250 output tokens per explanation → ~100 explanations/day on TPD — ample, because we explain lazily: top-20 ranked events pre-generated, the rest on click.

Cache key: (pair, TCA bucketed to a minute, verdict, data grade) in SQLite — repeated views cost zero tokens.

Fallback: a deterministic template renderer produces the same structure from the same record ("Groq down" can never break the demo). llama-4-scout configured as alternate model name.

5.6 What the LLM is *not* doing (anti-wrapper statement)

The FAR AWAY rubric penalizes thin wrappers. In OrbitGuard the LLM touches nothing upstream: propagation, screening, TCA, Pc, policy are pure physics/numerics validated against NASA tooling and a federal answer key. We will say this out loud in the README and the video.

6. TraCSS validation plan (the credibility engine)

Dataset (verified): five files — AerospaceIWDataset_20251009a.tar.gz (20.73 GB ephemerides), spherical answer key IVW_Releasable_Dataset_Spherical_DefaultHBR.csv.gz (198.8 MB), SFSH rectangular key IVW_Releasable_Dataset_SFSH_DiscreteHBR.csv.gz (62.4 MB), screening-volume mapping CSV, and the User's Guide PDF. CC0 license. Access via Google Form (immediate) — day-zero action #1. Disclaimer noted: dataset is for algorithm testing, not operational certification — we quote that disclaimer verbatim in our scorecard page (honesty, again).

Methodology:

1. Convert ephemerides → Parquet (one-time job; ~20.7 GB tar → columnar store, chunked).
2. Run our **Stage B + C engine directly on the provided ephemerides** (no SGP4 in the loop — the answer key tests *screening*, not propagation). Interpolate ephemeris points with the scheme the User's Guide prescribes (plan default: 5th-order Lagrange on position, derived velocity; confirm against the guide on day 2).
3. **Primary run:** spherical volumes / DefaultHBR key. Match events to the key on (object pair, TCA within tolerance τ ; τ from the User's Guide, default ± 5 s).
4. **Metrics:** recall (found/key), precision (found $\cap$ key/found), F1; TCA error distribution (RMS, p95); miss-distance error distribution; breakdowns by altitude regime and eccentricity; runtime + funnel-reduction stats on disclosed hardware.
5. **Stretch run:** SFSH rectangular volumes / DiscreteHBR key (per-object volumes via the mapping file — requires RIC-frame box tests in Stage C).
6. **Targets:** recall ≥ 0.98 , precision ≥ 0.95 on the spherical run; publish whatever we measure with error analysis (missed-event autopsies in an appendix). The commitment is to *publish*, not to a number we haven't measured yet.
7. Subset-first discipline: day-1 of validation uses a 1-day time slice to debug matching; full runs follow.

In-app Scorecard page renders the metrics JSON with the run configuration, dataset citation, git SHA, and hardware — a screenshot-ready artifact for the video, the README badge, and resumes.

Live-mode cross-check: our CelesTrak-driven screening list is diffed weekly against SOCRATES Plus output for overlapping pairs (independent oracle; differences investigated, not hidden).

7. Tech stack (final)

Layer	Choice	Notes
Language (engine)	Python 3.12	numpy, scipy; numba only if profiling demands it
Propagation	python-sgp4 (SatreArray)	C++-accelerated, vectorized; Vallado verification vectors in CI
Time/frames	Skyfield	TEME→GCRS→ITRF, UTC/TT/UT1, leap seconds
Spatial pairing	scipy.spatial.cKDTree	per-step range queries
Data store	Parquet (pyarrow) + SQLite	catalog snapshots, TraCSS ephemerides, explanation cache
API	FastAPI + uvicorn	OpenAPI contract frozen day 5 (frontend unblocks)

Layer	Choice	Notes
Scheduler	APScheduler	2-h CelesTrak sync; nightly Space-Track CDM pull
Frontend	Next.js 14 + TypeScript	
Globe	CesiumJS via Resium	Industry default for orbital viz (Wayfinder uses it); time-dynamic entities, free ion tier; deck.gl rejected (no native time-dynamic space frames); KeepTrack proves 30k objects feasible in-browser
Client propagation	satellite.js	smooth 60-fps animation without server round-trips
Plots	Plotly (encounter ellipse, funnel, scorecard)	the covariance-ellipse + HBR figure is the technical money shot
LLM	Groq llama-3.3-70b-versatile (alt: llama-4-scout)	Section 5.5 contract
Hosting (demo)	Vercel (web) + Fly.io/Render free (API) — plus fully-offline local mode	frozen-snapshot demo mode is the primary plan for the stage
CI	GitHub Actions	pytest (golden vectors, CARA Pc cases, sieve property tests), lint, type-check

8. Repository structure

```
orbitguard/
|-- README.md           # headline, scorecard badge, architecture, honest-limits section
|-- docs/
|   |-- VALIDATION.md   # TraCSS methodology + results (mirrors scorecard page)
|   |-- ALGORITHMS.md   # the math of Section 5, with references
|   `-- DEMO_SCRIPT.md
|-- engine/             # Python package (the data plane)
|   |-- ingest/         # celestrak.py, spacetrack.py, tracss.py, cache.py
|   |-- astro/          # propagate.py, frames.py, timescale.py
|   |-- screening/      # filters.py (Stage A), sieve.py (Stage B), tca.py (Stage C)
|   |-- risk/           # pc_foster.py, pc_chan.py, pc_max.py, policy.py
|   |-- explain/        # groq_client.py, prompts.py, templates.py, validator.py
|   |-- validation/     # tracss_harness.py, matching.py, metrics.py
|   `-- api/            # FastAPI app, schemas.py (pydantic = OpenAPI contract)
|-- web/                # Next.js app
|   |-- components/globe/ # CesiumViewer, OrbitEntities, ConjunctionMarkers
|   |-- components/inspector/ # EncounterPlane.tsx, EvidenceCard.tsx, ExplainPanel.tsx
|   |-- components/scorecard/
|   `-- pages/
|-- data/
|   |-- fixtures/        # frozen OMM subset + golden vectors (committed)
|   `-- (gitignored caches) # snapshots, TraCSS parquet
|-- tests/
|   |-- test_sgp4_golden.py # Vallado vectors, Skyfield cross-check
|   |-- test_sieve_property.py # randomized: no planted conjunction escapes the net
|   |-- test_pc_cara_cases.py # NASA CARA SDK test cases, ported
|   `-- test_policy.py
`-- .github/workflows/ci.yml
```

9. Phase plan (4–5 people, ~3 working weeks)

Minimum demoable path (protected at all costs): P0 spike → P1 → asset-vs-catalog screening on a cached snapshot → globe with conjunction markers → template explanations. Everything else upgrades this skeleton.

Phase	Days	Deliverable	Exit criterion
P0 — Day zero	D0–D1	Access requests (4.3), repo + CI scaffold, cached snapshot, e2e spike : propagate ISS, render one orbit in Cesium	One satellite moving on the globe from real data
P1 — Astro core	D2–D5	OMM ingest, vectorized SGP4, frames/time module, golden tests green	Catalog propagated on a grid; positions match references

Phase	Days	Deliverable	Exit criterion
P2 — Screening engine	D4–D8	Stages A/B/C, funnel metrics, property tests (planted conjunctions always found)	k=5 assets vs catalog, 7-day screen < 60 s laptop; SOCRATES cross-check sane
P3 — Risk + explain	D7–D11	Foster + Chan + PcMax ported; CARA test cases pass ; policy engine; Groq client + cache + fallback	UI-ready evidence records with verdicts and prose
P4 — Frontend	D6–D13	Globe (entities, timeline), conjunction list, Encounter Inspector (ellipse plot), scorecard page; starts D6 against the frozen OpenAPI contract with mocked data	Full click-through on mock + live data
P5 — TraCSS validation	D9–D14	Parquet conversion, harness, matching, metrics; spherical run complete; tuning; scorecard frozen	Published recall/precision + error autopsies
P6 — Polish + ship	D15–D18	Offline demo mode frozen, demo rehearsals, 2–3 min video, README/summary doc, commit-history hygiene pass	Round-1 deliverables submitted

Slack is built in: P3/P4/P5 overlap deliberately; if TraCSS access stalls (risk R1), P5 slides without blocking the demoable path.

10. Team split (5 lanes)

Lane	Owner	Scope	Depends on
A — Astrodynamics core	strongest math/physics person	P1: ingest, SGP4, frames/time, golden tests; consults on C	—
B — Screening & performance	algorithms/systems person	P2: sieve stages, vectorization, funnel, property tests	A (D5 interface: propagate(ids, t_grid) -> positions)
C — Risk & validation	numerics person	P3 Pc trio + policy; P5 TraCSS harness + metrics	A (frames), B (events)
D — Frontend & demo	UI/3D person	P4 globe, inspector, scorecard; P6 video editing	API contract (D5), mock-first
E — Platform & explainer	glue/devops person	ingest pipeline + caches, FastAPI, Groq layer, CI/CD, deployment, offline demo mode	everyone (owns the contract)

4-person variant: E merges into B (platform) and D (Groq layer). The graph has one deliberate chokepoint — the **OpenAPI contract frozen on D5** — after which frontend and engine evolve in parallel without coordination cost.

11. 3D globe & UX design

Views:

- 1. Mission Control (default).** Cesium globe; operator's assets in cyan with orbit trails; the ranked conjunction list docked right; DODGE/WAIT/WATCH cards color-coded (red/amber/green) with countdown-to-TCA. Cesium timeline scrubbing animates the encounter.
- 2. Encounter Inspector (the technical wow).** Split view: left — 3D flyby animation centered on the asset, both trails, closest-approach segment highlighted; right — the **encounter-plane plot**: combined covariance ellipse, hard-body circle, miss vector, with Pc/PcMax, the data- grade badge, and the evidence card; below — the Groq explanation with a "show raw evidence" toggle (the JSON itself: transparency as UI).
- 3. Validation Scorecard.** Recall/precision tiles, TCA/miss error histograms, the funnel strip, dataset citation + run SHA. Screenshot-ready.
- 4. Catalog context (ambient wow).** Background point-cloud of the full catalog (satellite.js, point primitives, LOD-decimated to keep 60 fps; KeepTrack demonstrates this scale works).

Performance budget: ≤ 5 polyline orbit entities + ~30k points (decimated to ~10k on weak GPUs); all heavy math server-side; client only animates.

Design language: dark navy space theme, mono typeface for numbers, color reserved exclusively for verdicts — the UI is the triage.

12. Demo script & video shot list

Live demo (3 minutes):

1. *Hook (20 s)*: "A university cubesat team gets a collision warning email. Cryptic format, 99% false-alarm base rate, and the tools that decode it cost more than their satellite. Here's what they should have instead." — globe already spinning.
2. *Screen (40 s)*: pick the Pixxel asset → screening funnel animates (30,000 → 9 → 1) with timings → ranked list appears; one amber WATCH card.
3. *Inspect (60 s)*: open the Encounter Inspector → flyby animation → encounter-plane ellipse → "this number, P_c , is computed with the same method NASA uses — and our implementation passes NASA CARA's own test cases" → Groq explanation reads out the verdict rationale.
4. *Trust (40 s)*: Scorecard page — "we didn't mark our own homework: X% recall against the official U.S. TraCSS answer key, 20 GB of federal test ephemerides" → honest-limits beat: "and when the data can't justify a number, we say so — TLE data can prove you're safe, never that you must move."
5. *Close (20 s)*: "Free, open source, built on public data, for the operators commercial SSA leaves behind." India angle: Pixxel/GalaxyEye/ISRO ecosystem.

Video (2–3 min) shot list: cold-open globe zoom (5 s) → problem stat over a real CDM screenshot (10 s) → funnel animation screen-capture (15 s) → inspector + ellipse (30 s) → scorecard with TraCSS citation (20 s) → architecture diagram pan (15 s) → honest-limits card (10 s) → team + repo URL (10 s). Record at 4K, dark theme, no music under voiceover segments.

13. Risk register

#	Risk	L×I	Mitigation
R1	TraCSS access approval delays	M× H	Day-zero Google Form (immediate path); email contact; P5 decoupled from demoable path; SOCRATES cross-check as fallback validation narrative
R2	20.7 GB dataset logistics	M× M	One download, checksum, Parquet conversion overnight, subset-first development
R3	Frame/time bugs (TEME/UTC traps) silently corrupt everything	M× H	Single audited frames module; golden vectors in CI; Skyfield cross-check; Foster-vs-Chan disagreement alarm
R4	P_c implementation wrong	M× H	Port NASA CARA test cases; two independent methods cross-checked in CI
R5	Screening misses events (sieve bug)	L×H	Property tests with planted conjunctions; the pad guarantee is provable; TraCSS recall measures it
R6	$O(N^2)$ ambition creep (all-vs-all)	M× M	Product-scoped to asset-vs-catalog; all-vs-all is stretch-only
R7	Live demo flakiness (APIs, network)	H× H	Frozen-snapshot offline mode is the primary demo path ; live mode is the encore
R8	Groq outage / rate limit	M×L	Response cache + deterministic template fallback; explainer is decorative to correctness
R9	CelesTrak IP block from over-fetching	M× M	2-h cadence respected in code (not convention); ETag/timestamp checks; single shared cache
R10	Catalog rollover mid-event (~12 July)	M× M	OMM-first architecture (Section 4.2); integer IDs end-to-end
R11	Team integration crunch	M× H	OpenAPI contract frozen D5; mock-first frontend; weekly integration checkpoints

"Deep vs. shallow globe" test (rubric mapping): Technical depth = sieve guarantee + Foster/ Chan/ P_c Max math + CARA test parity + frames discipline. Execution = TraCSS scorecard + CI + funnel telemetry. Originality = validation-in-the-open + explainable triage + prove-safety-not-danger policy. Impact = priced-out operators, India story. Design = inspector ellipse + verdict-first UI. Every axis has a named artifact a judge can click.

14. Stretch goals (only after P6 is safe)

1. **SFSH rectangular-volume validation** (second answer key, RIC box screening).
2. **Live CDM auto-ingest** from Space-Track with rigorous Pc on covariance pairs.
3. **All-vs-all nightly batch** (SOCRATES-style) with published runtime engineering notes.
4. **Maneuver what-if:** $\pm$ along-track Δv slider $\rightarrow$ re-screened miss distance/Pc in the inspector.
5. **Alerting:** email/webhook on new WATCH+ events for subscribed assets.
6. **GPU propagation** (CuPy/jax port of the grid stage; literature exists — jaxsgp4).

15. Resume bullets (write them now, build to make them true)

*Built **OrbitGuard**, a free browser-based satellite conjunction-assessment system: vectorized SGP4 screening of operator assets against the 30,000-object public catalog with a provable coarse-grid sieve, 2D encounter-plane collision probability (Foster/Chan) whose implementation **passes NASA CARA's published test cases**, and worst-case PcMax bounds when covariance is unavailable — **validated at X% recall against the official U.S. TraCSS conjunction answer key** (20 GB federal ephemeris dataset), with an LLM explanation layer under a strict narrate-only contract. Python/FastAPI · CesiumJS · Groq.*

Per-lane variants: lane A leads with frames/SGP4 golden-testing; lane B with the sieve guarantee + performance funnel; lane C with Pc math + federal-dataset validation; lane D with time-dynamic 3D UX; lane E with the grounded-LLM contract + zero-cost infra.

16. Consolidated day-zero checklist

- ☐ TraCSS Google Form submitted (+ Drive request as backup) — owner: C
- ☐ Space-Track account + API throttle rules read — owner: E
- ☐ Groq key created, llama-3.3-70b-versatile quota confirmed — owner: E
- ☐ Cesium ion free token — owner: D
- ☐ Celestrak active OMM JSON snapshot cached + fixture subset committed — owner: B
- ☐ TraCSS 20.7 GB download started + checksums — owner: C
- ☐ Demo asset set chosen (Pixxel / GalaxEye / ISRO / ISS) — owner: A
- ☐ Repo scaffold + CI + Conventional Commits convention agreed — owner: E

17. References

Data & standards

1. Office of Space Commerce — Dataset for Conjunction Assessment Verification.
<https://space.commerce.gov/dataset-for-conjunction-assessment-verification/>
2. Office of Space Commerce — TraCSS publishes "Dataset for Conjunction Assessment Verification".
<https://space.commerce.gov/tracss-publishes-dataset-for-conjunction-assessment-verification/>
3. TraCSS CDM Specification v2.1 (2026).
https://space.commerce.gov/wp-content/uploads/2026/01/TraCSS-Spec-001-v2.1_CDM.pdf
4. Celestrak — A New Way to Obtain GP Data (formats, cadence, rate limits, >99999 catalog IDs).
<https://celestrak.org/NORAD/documentation/gp-data-formats.php>
5. Space-Track.org — API documentation (GP classes, CDM access). <https://www.space-track.org/documentation>
6. CCSDS 502.0-B-3 — Orbit Data Messages (OMM); CCSDS Conjunction Data Message (508.0) draft.
<https://ccsds.org>

7. CelesTrak SATCAT documentation — catalog-number exhaustion at 69999 (~2026-07-12).

<https://celestrak.org/satcat/satcat-format.php>

Propagation & TLE accuracy

8. Vallado, Crawford, Hujak, Kelso — Revisiting Spacetrack Report #3 (AIAA 2006-6753; SGP4 verification vectors, TEME frame).
9. Levit & Marshall — Improved orbit predictions using two-line elements. arXiv:1002.2277.
10. Oltrogge — Parametric Characterization of SGP4 Theory and TLE Positional Accuracy (AMOS 2014). <https://amostech.com/TechnicalPapers/2014/Poster/OLTROGGE.pdf>
11. How long can you trust a Starlink TLE? (empirical SGP4 vs operator truth, 2026). arXiv:2605.19850.
12. Rhodes — python-sgp4; Skyfield. <https://github.com/brandon-rhodes/python-sgp4> · <https://rhodesmill.org/skyfield/>

Screening filters

13. Hoots, Crawford, Roehrich — An analytic method to determine future close approaches between satellites. Celestial Mechanics 33 (1984) 143–158.
14. Alarcón-Rodríguez, Martínez-Fadrique, Klinkrad — Collision risk assessment with a "smart sieve" method. ESA SP-486 (2002).
15. AGI — A Description of Filters for Minimizing the Time Required for Orbital Conjunction Computations. <https://www.agi.com/getmedia/d56ede5a-efe7-48de-af8b-c122117496af/A-Description-of-Filters-for-Minimizing-the-Time-Required-for-Orbital-Conjunction-Computations.pdf>
16. A novel conjunction filter based on the minimum distance between perturbed trajectories (2024). arXiv:2410.20928.

Collision probability

17. Foster & Estes — A parametric analysis of orbital debris collision probability... NASA JSC-25898 (1992).
18. Chan — Spacecraft Collision Probability. The Aerospace Press (2008); original method 1997.
19. Patera — General method for calculating satellite collision probability. JGCD 24(4) (2001).
20. Alfano — A numerical implementation of spherical object collision probability. J. Astronaut. Sci. 53(1) (2005).
21. Hejduk et al. — NASA Robotic CARA Probability of Collision. NASA NTRS 20190011726. <https://ntrs.nasa.gov/api/citations/20190011726/downloads/20190011726.pdf>
22. CARA — Implementation Recommendations and Usage (2D-Pc). NASA NTRS 20190028900. <https://ntrs.nasa.gov/api/citations/20190028900/downloads/20190028900.pdf>
23. nasa/CARA_Analysis_Tools — open-source SDKs with test cases (Pc2D Foster, max Pc, covariance tools). https://github.com/nasa/CARA_Analysis_Tools
24. Maximum collision probability considering variable size, shape, and orientation of covariance ellipse (Alfano 2014 / Frisbee 2015 lineage). Adv. Space Research. <https://www.sciencedirect.com/science/article/abs/pii/S0273117716302733>
25. Hejduk — Covariance Manipulation for Conjunction Assessment. NASA NTRS 20160010497.
26. A review of space-object collision probability computation methods. https://file.sciopen.com/sciopen_public/1529721751973974018.pdf

Operations & thresholds

27. NASA Spacecraft Conjunction Assessment and Collision Avoidance Best Practices Handbook, v2 (2023). NASA NTRS 20230002470. <https://ntrs.nasa.gov/citations/20230002470>

Prior art & visualization

28. Kelso — SOCRATES: Satellite Orbital Conjunction Reports Assessing Threatening Encounters in Space (AAS 05-124). <https://celestrak.org/SOCRATES/>
29. Cesium — Wayfinder Tracks Orbital Debris with CesiumJS (2023). <https://cesium.com/blog/2023/04/18/wayfinder-tracks-orbital-debris-with-cesiumjs/>
30. KeepTrack.space — open-source 3D satellite toolkit. <https://keeptrack.space>
31. satellite.js — SGP4 in JavaScript. <https://github.com/shashwatak/satellite-js>

LLM infrastructure

32. Groq — Rate Limits documentation (free-tier RPM/RPD/TPM/TPD). <https://console.groq.com/docs/rate-limits>

End of plan. First task after team sign-off: the Section 16 checklist, then the P0 spike — one satellite moving on the globe from real data within 24 hours.